INDIAFY PLATFORM QA AUDIT

SEPTEMBER 2026 | ENTERPRISE EDITION v2.5.0

Admin Panel: Complete Forensic Audit & Defect Report

An exhaustive static code and architectural audit inspecting 21 administrative dashboard pages, API communication interceptors, authorization guards, database persistence models, orphan unrouted components, silent notification failures, mock data fallbacks, and visual UI/UX defects.

21	3	4	6	13	P0
Total Admin Pages	Completely Unrouted	Nav Link Mismatches	Critical Backend Bugs	Silent Toasts / Glitches	Overall Action Level

1. Executive Summary & Audit Scope

Target Application: IndiaFy Multi-Vendor Hyperlocal & B2B E-Commerce Marketplace
Audited Subsystem: Administrative Control Panel (React/Vite SPA + Express/Node.js API)
Audit Objective: Detect all software bugs, logic flaws, broken/unrouted views, dead code paths, security session gaps, and visual glitches preventing production deployment.

Core Architectural Vulnerabilities Discovered:
• **Orphan Unrouted Views:** Three complete pages (CreateCustomer.jsx, CreateOrder.jsx, and WhatsappAutomation.jsx) exist in the codebase but have never been registered in App.jsx.
• **Broken Navigation Links:** Coupons and Inventory pages are defined in routes but completely omitted from Sidebar.jsx. Additionally, OrderManagement.jsx table rows have no click handlers to view individual order details.
• **Critical Data Corruption:** The system settings update endpoint spreads string values into objects, corrupting brand name and logo URLs.
• **Pretended Database Operations:** Customer status blocking literally contains a developer comment 'pretending' to save, failing to persist status.
• **Complete Notification Blackout:** 13 admin pages trigger react-toastify toasts, but App.jsx only mounts react-hot-toast. Consequently, 100% of administrative action feedback (approvals, rejections, deletes, saves) fails silently without user notification.

Category	Audited Surface	Identified Status
Frontend Routes & SPA	frontend/src/App.jsx, Sidebar.jsx, Header.jsx	**3 Unrouted, 4 Missing Links**
Admin Core Pages	21 JSX components in frontend/src/pages/admin/	**Mock Traps, Dead Buttons, Broken Media**
API Interceptors & Auth	axiosInstance.js, adminAuthStore.js, ProtectedRoute.jsx	**Token Stripping, Missing Startup Validation**
Backend Controllers	controllers/admins/management.controllers.js, auth.controllers.js	**String Spread Bug, Fake DB Save, 404 CastError**

2. Master Audit Findings Matrix

ID	Type	Target Location	Defect Summary	Severity
#UR-01	Unrouted	pages/admin/CreateCustomer.jsx	Omitted from App.jsx and Sidebar; inputs lack state bindings, buttons have no handlers	**HIGH**
#UR-02	Unrouted	pages/admin/CreateOrder.jsx	Omitted from App.jsx and Sidebar; hardcoded dummy customer 'Julian Hakes'; dead modal	**HIGH**
#UR-03	Unrouted	pages/admin/WhatsappAutomation.jsx	Omitted from App.jsx and Sidebar; signs templates with third-party '– Team Graphura'	**MEDIUM**
#UR-04	Unlinked Nav	pages/admin/Coupons.jsx	Routed in App.jsx but missing from Sidebar.jsx; 100% hardcoded mock data, no backend API	**HIGH**
#UR-05	Unlinked Nav	pages/admin/Inventory.jsx	Missing from Sidebar.jsx; only hardcoded for 1 blazer product; broken 404 image link	**HIGH**
#UR-06	Broken UX	pages/admin/OrderDetail.jsx	OrderManagement table rows lack navigation links to open individual order detail views	**HIGH**
#UR-07	Nav / Data	pages/admin/Profile.jsx	Route is /admin/profiles vs /admin/profile; mock data with admin@graphura.com & fake logout	**MEDIUM**
#BG-01	Data Corrupt	management.controllers.js:1046	String spreading bug in updateSystemSettings corrupts brandName, logoUrl, and faviconUrl	**CRITICAL**
#BG-02	Fake Save	management.controllers.js:361	updateCustomerStatus literally 'pretends' to update; customer block state is never persisted	**CRITICAL**
#BG-03	Silent Fail	App.jsx & 13 Admin Pages	Missing : all react-toastify alerts across 13 admin pages fail silently	**CRITICAL**
#BG-04	Auth Token	frontend/src/utils/axiosInstance.js	Admin token not sent on non-/admin shared endpoints (e.g. /orders/:id) causing 401/403	**CRITICAL**
#BG-05	Session Bug	frontend/src/App.jsx:245	fetchMe() never called for admin on app startup; admin session unvalidated on page refresh	**HIGH**
#BG-06	API Crash	pages/admin/SupportInbox.jsx:157	Fallback mock tickets have IDs '1' and '2'; replying triggers 404/500 Mongoose CastError	**HIGH**
#BG-07	Blank Screen	pages/admin/RoleManagement.jsx	If AdminRole collection is empty, UI renders completely blank without fallback system roles	**HIGH**
#BG-08	Mock Trap	management.controllers.js:802	getFinancialStats falls back to INR 14.82L revenue & fake transactions if paid orders total is 0	**MEDIUM**
#GL-01	Broken 404	pages/admin/Inventory.jsx:81	Broken image: Jaket1.webp misspelled (file is Jacket1.webp) and unimported in Vite bundle	**MEDIUM**
#GL-02	Vendor Copy	Profile.jsx & WhatsappAutomation.jsx	Exposes uncleaned third-party branding ('Team Graphura' and 'admin@graphura.com')	**MEDIUM**
#GL-03	Dead Filter	pages/admin/Dashboard.jsx:262	Timeframe buttons (Today, Weekly, Monthly, Yearly) do not trigger backend API queries	**LOW**
#GL-04	RBAC Lock	components/ProtectedRoute.jsx:88	Only checks 'admin' or 'super_admin'; specialized roles (Product Admin) locked out	**HIGH**

3. Deep-Dive: Unrouted Systems & Missing Navigation (Part 1)

#UR-01 — CreateCustomer.jsx — Completely Unrouted with Non-Functional Form

Location: frontend/src/pages/admin/CreateCustomer.jsx

Description: CreateCustomer.jsx is physically located in the admin pages directory but has zero references in App.jsx or Sidebar.jsx. It is completely unreachable via URL or navigation.
Technical Flaws: Form inputs have no value or onChange state handlers. The 'Save Customer' and 'Save & Create Order' buttons have no onClick handlers or API calls. The customer profile avatar drag-and-drop only creates a local blob URL without server upload capability.
Remediation: Define route /admin/customers/create in App.jsx, link it from a new 'Add Customer' button in CustomerManagement.jsx, and implement backend customer creation API.

#UR-02 — CreateOrder.jsx — Completely Unrouted with Static Mock Customer

Location: frontend/src/pages/admin/CreateOrder.jsx

Description: CreateOrder.jsx is totally omitted from the application routing layer.
Technical Flaws: Contains hardcoded customer identity ('Julian Hakes', 'aamirf.khan@gmail.com'). The product selector modal renders dummy sample products ('$49.00') where clicking 'Add to Order' does nothing. 'Save Draft' and 'Create Order & Notify Customer' buttons have no event handlers or API integration.
Remediation: Route to /admin/orders/create, link from OrderManagement.jsx, and connect to live product search and customer autocomplete APIs.

#UR-03 — WhatsappAutomation.jsx — Unrouted Third-Party Vendor Remnant

Location: frontend/src/pages/admin/WhatsappAutomation.jsx

Description: This component is completely unrouted and unused in the project.
Technical Flaws: All five message templates are signed with '**– Team Graphura**', revealing uncleaned boilerplate copied from an external project. The message content textarea is readOnly, the 'Save Template' button is dead, and message logs display a single hardcoded row for 'Rahul Sharma'.
Remediation: Purge this file or rebrand templates to IndiaFy and integrate with a WhatsApp Business API service.

#UR-04 — Coupons.jsx — Missing from Admin Sidebar Navigation

Location: frontend/src/pages/admin/Coupons.jsx & components/admin/Sidebar.jsx

Description: App.jsx routes /admin/coupons to Coupons.jsx, but Sidebar.jsx omits the Coupons link entirely from its navigation items. An administrator cannot access coupon management via UI.
Technical Flaws: The page uses 100% hardcoded client-side mock data ('LUXE2024', 'WELCOME10', 'FESTIVE30'). Although coupon.model.js exists in the backend models, there are zero routes or controllers for coupons.
Remediation: Add Coupons to Sidebar.jsx under 'Commerce & Users' and implement backend coupon CRUD APIs.

3. Deep-Dive: Unrouted Systems & Missing Navigation (Part 2)

#UR-05 — Inventory.jsx — Missing from Sidebar & Hardcoded to Single Product

Location: frontend/src/pages/admin/Inventory.jsx & components/admin/Sidebar.jsx

Description: /admin/inventory is routed in App.jsx but missing from the sidebar. Only a notification dropdown link in Header.jsx points to it.
Technical Flaws: Rather than managing platform-wide inventory, the page is hardcoded for a single product: 'Silk Oversized Blazer' (SKU: BLAZ-SILK-001). Stock adjustments, variant creations, and logs are all static client arrays.
Remediation: Add Inventory to Sidebar.jsx and connect it to a multi-product warehouse inventory endpoint.

#UR-06 — OrderDetail.jsx — Unreachable via Order Management Table

Location: frontend/src/pages/admin/OrderManagement.jsx & OrderDetail.jsx

Description: OrderDetail.jsx exists and is routed to /admin/orders/:id, but the orders table in OrderManagement.jsx has no clickable link or navigation handler to view individual orders.
Technical Flaws: useNavigate is imported in OrderManagement.jsx but never invoked. Administrators cannot inspect customer shipping addresses, line items, or customer notes from the table.
Remediation: Add an onClick={() => navigate(`/admin/orders/${o._id}`)} to table rows and Order IDs.

#UR-07 — Profile.jsx — Plural Route Mismatch & Disconnected Auth Data

Location: frontend/src/pages/admin/Profile.jsx & Header.jsx

Description: Route is defined as /admin/profiles (plural) in App.jsx and Header, diverging from standard convention (/admin/profile). The sidebar has no profile link.
Technical Flaws: Does not load authenticated admin data from useAdminAuthStore. Hardcodes 'Aamir Farid' and 'admin@graphura.com'. The logout button triggers a dummy alert('Logged out'). 'Save Changes' and 'Change Password' buttons have no implementation.
Remediation: Normalize route to /admin/profile, bind with useAdminAuthStore, and wire real logout.

Recommended Administrative Route Hierarchy Architecture

To maintain an intuitive, production-grade navigation experience, the routing layout in App.jsx and Sidebar.jsx should be unified into three logical tiers:

- **Primary Hub:** Dashboard (/admin/dashboard), Analytics (/admin/analytics), Profile (/admin/profile)
- **Marketplace Operations:** Customers (/admin/customers, /create), Sellers (/admin/active-sellers, /pending-applications), Stores (/admin/stores)
- **Order & Catalog:** Products (/admin/products), Categories (/admin/categories), Orders (/admin/orders, /:id, /create), Coupons (/admin/coupons), Inventory (/admin/inventory)
- **Governance & Finance:** Payments (/admin/payments), Support Tickets (/admin/tickets), Role Governance (/admin/roles), Audit Trail (/admin/audit-logs), Settings (/admin/settings)

4. Critical Code Bugs & Runtime Errors (Part 1)

#BG-01 — String Spreading Bug Corrupts Global System Settings

CRITICAL

Location: backend/controllers/admins/management.controllers.js (Lines 1043-1048)

Root Cause: In updateSystemSettings, the controller iterates over keys and executes:
settings[key] = { ...settings[key], ...req.body[key] };
However, the keys array includes brandName, logoUrl, and faviconUrl, which are **scalar strings**, not JavaScript objects!
When an object spread is applied to a string: { ..."Indiafy" }, JavaScript creates an object of indexed characters: { '0': 'I', '1': 'n', '2': 'd', ... }. Assigning this to a Mongoose String schema field throws a **CastError** or permanently corrupts the database document.

Vulnerable Code Snippet:

```
// BUGGY CODE (management.controllers.js:1044)
const keys = ['brandName', 'logoUrl', 'faviconUrl', 'contactDetails', 'payments', 'email', 'security', 'commissions'];
for (const key of keys) {
if (req.body[key] !== undefined) {
settings[key] = { ...settings[key], ...req.body[key] }; // <-- Spreading strings into character objects!
}
}
```

Remediation Patch:

```
// RECOMMENDED FIX
const objectKeys = ['contactDetails', 'payments', 'email', 'security', 'commissions'];
const scalarKeys = ['brandName', 'logoUrl', 'faviconUrl'];
for (const key of scalarKeys) {
if (req.body[key] !== undefined) settings[key] = String(req.body[key]).trim();
}
for (const key of objectKeys) {
if (req.body[key] !== undefined) settings[key] = { ...settings[key], ...req.body[key] };
}
```

#BG-02 — updateCustomerStatus Literally 'Pretends' to Save in Database

CRITICAL

Location: backend/controllers/admins/management.controllers.js (Lines 360-363)

Root Cause: When an administrator blocks or unblocks a customer, the backend controller literally states:
// In a real DB, we would add the field isBlocked to customer model if missing
// For now, let's pretend we update the profile metadata
No customer.save() or DB update is ever executed! Furthermore, getCustomerList (Line 329) always hardcodes isBlocked: false. Blocked abusive customers are never persisted and can continue logging in.

Vulnerable Code Snippet:

```
// BUGGY CODE (management.controllers.js:360)
// For now, let's pretend we update the profile metadata
const before = { isBlocked: !isBlocked };
const after = { isBlocked };
await logAdminAction(req, 'UPDATE_CUSTOMER_STATUS', `customer:${id}`, before, after);
return res.status(200).json(new ApiResponse(200, { id, isBlocked }, 'Updated')); // Zero DB writes!
```

Remediation Patch:

```
// RECOMMENDED FIX
// 1. Add isBlocked: { type: Boolean, default: false } to backend/models/customers/auth.model.js
customer.isBlocked = Boolean(isBlocked);
await customer.save();
// 2. In customer login controller, verify !customer.isBlocked before issuing token.
```

4. Critical Code Bugs & Runtime Errors (Part 2)

#BG-03 — Silent Notification Failure: Missing ToastContainer Across Entire App

CRITICAL

Location: frontend/src/App.jsx & 13 Admin Components

Root Cause: All 13 major admin pages import toast from react-toastify.
However, App.jsx only mounts <Toaster /> from react-hot-toast! There is no <ToastContainer /> mounted anywhere in the application component tree.
Impact: 100% of administrative feedback notifications (seller approval, rejection, product deletion, status change, settings saved, ticket reply) fail silently. The user receives zero visual confirmation after performing critical actions.

Vulnerable Code Snippet:

```
// In 13 admin pages:
import { toast } from 'react-toastify';
toast.success('Seller approved successfully'); // <-- Completely silent! Nothing appears in DOM!

// In App.jsx:
import { Toaster } from 'react-hot-toast'; // Only react-hot-toast container is mounted.
```

Remediation Patch:

```
// RECOMMENDED FIX in frontend/src/App.jsx:
import { ToastContainer } from 'react-toastify';
import 'react-toastify/dist/ReactToastify.css';
// Inside App component JSX:
```

#BG-04 — Admin Authorization Token Stripped on Shared API Endpoints

CRITICAL

Location: frontend/src/utils/axiosInstance.js (Lines 82-95)

Root Cause: axiosInstance.js determines which token to inject based strictly on URL sub-strings:
const isAdminRoute = url.includes('/admin');
When an administrator accesses shared routes that do not contain /admin in the URL (such as OrderDetail.jsx calling /orders/${id}), isAdminRoute evaluates to false. The interceptor then falls back to searching for a seller or customer token, causing **401 Unauthorized** or **403 Forbidden**.

Vulnerable Code Snippet:

```
// BUGGY CODE (axiosInstance.js:82)
if (isAdminRoute) {
token = getAdminToken();
} else if (isCustomerRoute) {
token = getCustomerToken();
} else {
token = getSellerToken() || getCustomerToken(); // Admin token completely ignored on shared endpoints!
}
```

Remediation Patch:

```
// RECOMMENDED FIX in axiosInstance.js:
const currentPath = typeof window !== 'undefined' ? window.location.pathname : '';
const isAdminContext = currentPath.startsWith('/admin') || url.includes('/admin');
if (isAdminContext && getAdminToken()) {
token = getAdminToken();
} else if (isSellerRoute || currentPath.startsWith('/seller')) {
token = getSellerToken() || getCustomerToken();
} else { token = getCustomerToken(); }
```

4. Critical Code Bugs & Runtime Errors (Part 3)

#BG-05 — Admin Session Never Initialized or Validated on App Startup

HIGH

Location: frontend/src/App.jsx (Lines 245-250)

Root Cause: When the application boots or reloads, App.jsx only runs:
Promise.allSettled([fetchCustomer('customer'), fetchSeller('seller')])
The admin store's fetchMe() is never called! If an admin reloads the page, their session is never verified against the backend. Stale or revoked admin tokens remain active until an API call fails.

Vulnerable Code Snippet:

```
// BUGGY CODE (App.jsx:245)
Promise.allSettled([
fetchCustomer('customer'),
fetchSeller('seller'),
// fetchAdmin is completely absent!
]).finally(() => setAuthReady(true));
```

Remediation Patch:

```
// RECOMMENDED FIX in App.jsx:
const { fetchMe: fetchAdmin } = useAdminAuthStore();
Promise.allSettled([
fetchCustomer('customer'),
fetchSeller('seller'),
fetchAdmin(),
]).finally(() => setAuthReady(true));
```

#BG-06 — Support Ticket Crash on Empty State with Dummy IDs '1' and '2'

HIGH

Location: frontend/src/pages/admin/SupportInbox.jsx (Lines 156-167)

Root Cause: When no tickets exist in database (tickets.length === 0), SupportInbox.jsx renders two hardcoded dummy tickets with IDs _id: '1' and _id: '2'.
When an administrator clicks a ticket and sends a reply, an API call is dispatched to /admin/management/tickets/1/reply. Mongoose fails to cast string '1' to a valid 24-character hex **ObjectId**, throwing a **500 Server Error**.

Vulnerable Code Snippet:

```
// BUGGY CODE (SupportInbox.jsx:157)
safeTicketsList.length === 0 ? (
// Invalid ObjectId!
) : (...)
```

Remediation Patch:

```
// RECOMMENDED FIX in SupportInbox.jsx:
safeTicketsList.length === 0 ? (

No support tickets currently in queue.

) : ( ... )
```

#BG-07 — Role Management Blank Page When AdminRole Collection Empty

HIGH

Location: frontend/src/pages/admin/RoleManagement.jsx & management.controllers.js:1072

Root Cause: getRoles returns AdminRole.find({}). On clean or unseeded databases, this returns an empty array. RoleManagement.jsx provides no fallback system roles, resulting in an entirely blank screen.
Remediation: Auto-seed default roles (SUPER_ADMIN, OPERATIONS_MANAGER, SUPPORT_MANAGER) from permissionGuard.middleware.js.

Vulnerable Code Snippet:

```
// BUGGY CODE (management.controllers.js:1072)
const roles = await AdminRole.find({});
return res.status(200).json(new ApiResponse(200, roles, 'Access roles loaded')); // Returns [] on unseeded DB!
```

Remediation Patch:

```
// RECOMMENDED FIX in management.controllers.js:
let roles = await AdminRole.find({});
if (!roles.length) {
roles = await seedDefaultAdminRoles();
}
return res.status(200).json(new ApiResponse(200, roles, 'Access roles loaded'));
```

5. UI Glitches, Broken Media & Mock Traps

Broken 404 Image in Inventory `pages/admin/Inventory.jsx:81`

Line 81 uses 'src=../../assets/products/women/Jaket1.webp'. The actual filename on disk is 'Jacket1.webp' (with a 'c'). Moreover, Vite does not bundle relative string URLs unless imported or placed in /public, rendering an ugly 404 broken image icon on screen.

Third-Party Vendor Remnants ('Team Graphura') `Profile.jsx & WhatsappAutomation.jsx`

Profile.jsx hardcodes email 'admin@graphura.com' and name 'Aamir Farid', while WhatsappAutomation.jsx signs all message templates with '– Team Graphura'. This exposes uncleaned boilerplate copied from an external project.

Non-Functional Dashboard Timeframe Tabs `pages/admin/Dashboard.jsx:262`

The timeframe toggle buttons ('Today', 'Weekly', 'Monthly', 'Yearly') only update a local state variable 'activeFilter'. They do not query the backend or filter the revenue chart.

Overly Rigid ProtectedRoute Role Guard `components/ProtectedRoute.jsx:88`

The guard checks if adminAuth.user?.role?.toLowerCase() === 'admin' || 'super_admin'. If a specialized staff member logs in with role 'Product Admin', 'Support Admin', or 'Operations Manager', they are blocked and redirected to /admin/login.

Dead Buttons Across Administrative Suite `Multiple Admin Components`

Multiple buttons have empty or missing onClick handlers: Coupons.jsx ('Edit', 'Disable'), Inventory.jsx ('New Variant', 'Update Stock Level'), CreateCustomer.jsx ('Save Customer'), CreateOrder.jsx ('Save Draft', 'Create Order'), Profile.jsx ('Save Changes', 'Change Password').

Admin Email Case Sensitivity on Login `middlewares/emailPresent.middleware.js:29`

Signup converts email to lowercase, but login's admin middleware does not sanitize or lowercase the input email. If the user enters an email with capitalized letters or spaces, it returns a false 404 'Email is not found'.

Financials Endpoint Fallback Trap `controllers/admins/management.controllers.js:802`

When total revenue is 0 or orders are empty, getFinancialStats falls back to INR 14,82,000 and 2 fake transactions. This causes the UI to show fake transactions, but clicking 'Export Ledger' alerts 'No transaction data available'.

Dark Mode Inconsistencies `Coupons, CreateCustomer, CreateOrder, Profile`

These components use hardcoded white background containers ('bg-white', 'text-black') inside a dark gradient page background, creating jarring contrast issues in dark theme.

6. Prioritized Remediation Roadmap & Implementation Plan

Phase	Priority	Action Items (Technical Fixes)	Impact & Outcome
Phase 1	**P0 (Immediate)**	1. Fix string spreading bug in updateSystemSettings. 2. Mount <ToastContainer /> in App.jsx so all admin action feedback displays. 3. Fix token injection in axiosInstance.js for non-/admin endpoints (e.g. /orders/:id). 4. Add fetchAdmin() in App.jsx startup hook. 5. Save isBlocked in DB in updateCustomerStatus and check during customer login.	Prevents data corruption, crashes & 401s; restores user feedback
Phase 2	**P1 (High)**	1. Add Coupons link to Sidebar.jsx & implement coupon CRUD in backend. 2. Make OrderManagement.jsx table rows clickable to navigate to OrderDetail.jsx. 3. Remove mock IDs '1' and '2' in SupportInbox.jsx and replace with clean empty states. 4. Wire Profile.jsx to useAdminAuthStore and connect real logout. 5. Route or clean up CreateCustomer.jsx and CreateOrder.jsx.	Restores missing navigation flows & eliminates dead-ends
Phase 3	**P2 (Polish)**	1. Fix broken image URL in Inventory.jsx (rename to Jacket1.webp). 2. Purge all leftover 'Team Graphura' strings across templates. 3. Connect Dashboard timeframe filter buttons to backend aggregations. 4. Unify dark mode theme classes across all admin components.	Delivers a flawless, production-ready enterprise aesthetic

Audit Sign-off & Verification Status

STATUS: AUDIT COMPLETED & VERIFIED

This forensic technical audit report has been automatically generated after complete source code analysis across the frontend React components, Zustand stores, Axios interceptors, Express controllers, route definitions, and Mongoose schemas. All findings represent actionable defects currently present in the repository.

Audit Engineer: Antigravity AI Engineering Assistant (Advanced Agentic Architecture)
Platform: IndiaFy Enterprise Hyperlocal Multi-Vendor Ecosystem
Generated File: IndiaFy_Admin_Panel_Audit_Report.pdf